

Python Functions – Complete Notes with Assignments

What is a Function?

A **function** is a **set of instructions** or an **independent block of code** that performs a specific task.

- Improves code reusability
 - Makes program structured
 - Reduces repetition
-

◆ TYPES OF FUNCTIONS

Built-in Functions

These are already provided by Python.

Examples:

- `len()`
- `max()`
- `min()`
- `sum()`

- `type()`
- `print()`
- `reverse()`
- `upper()`
- `lower()`
- `title()`

Example:

```
print(len("PYTHON"))
```

Output:

```
6
```

```
print(max(10, 40, 20))
```

Output:

```
40
```



Assignment 1

1. Find the length of `"ScholarLogic"`.
2. Find maximum from: 15, 99, 23, 67.
3. Convert `"python programming"` into uppercase.
4. Reverse a list `[10, 20, 30, 40]`.

User Defined Functions

When the **user creates their own function**, it is called a user-defined function.

 **Example:**

```
def addition(a, b):  
    print(a + b)
```

```
addition(5, 10)
```

Output:

```
15
```

Assignment 2

1. Create a function to subtract two numbers.
 2. Create a function to print "Welcome to Python Class".
 3. Create a function to multiply two numbers.
-

Function Without Arguments

A function that does not take parameters.

```
def greet():  
    print("Hello")
```

```
greet()
```



Assignment 3

1. Create a function that prints today's day.
 2. Create a function that prints your name.
-

4 Function With Arguments

Function that accepts parameters.

```
def square(n):  
    print(n * n)
```

```
square(4)
```

Output:

```
16
```



Assignment 4

1. Create a function to find cube of a number.
 2. Create a function to find area of rectangle.
 3. Create a function to calculate simple interest.
-

5 Function Without Return

It prints output but does not return value.

```
def message():  
    print("No return value")
```

```
message()
```

Function With Return (Very Important)

```
def get_number():  
    return 10  
  
number = get_number()  
print(number)
```

Output:

10

Flow Explanation:

- Function is called
 - `return 10`
 - Value goes back to caller
 - Stored in variable
 - Printed
-

Assignment 5

1. Create a function that returns square of number.
 2. Create a function that returns maximum of two numbers.
 3. Create a function that returns product of list elements.
-

Default Argument Function

```
def greet(name="Student"):  
    print(name)
```

```
greet()
```

Output:

```
Student
```

```
def power(base, exp=2):  
    print(base ** exp)
```

```
power(5)
```

Output:

```
25
```



Assignment 6

1. Create function with default country = "India".
 2. Create function to calculate power with default exponent = 3.
-

8 Keyword Arguments

```
def student(name, age):  
    print(name, age)  
  
student(age=21, name="Ravi")
```

Assignment 7

1. Create login function with username and password using keyword arguments.
 2. Create function to display employee details.
-

9 Variable Length Arguments (*args)

Used when we don't know number of arguments.

```
def total(*nums):  
    print(sum(nums))  
  
total(10, 20, 30)
```

Output:

```
60
```

Assignment 8

1. Create function to find maximum using *args.
2. Create function to multiply all numbers using *args.

Keyword Variable Arguments (**kwargs)

```
def details(**info):  
    print(info)  
  
details(name="Raju", city="Chennai")
```

Output:

```
{'name': 'Raju', 'city': 'Chennai'}
```



Assignment 9

1. Create profile function using **kwargs.
 2. Create function to store product details dynamically.
-

Nested Function

Function inside another function.

```
def outer():  
    def inner():  
        print("Inside inner")  
    inner()  
  
outer()
```



Assignment 10

1. Create parent and child function.
 2. Try calling inner function outside. What happens?
-



12 Closure (Very Important ★★)

Calling inner function outside using return.

```
def outer():  
    def inner():  
        print("Inside Function")  
    return inner
```

```
x = outer()  
x()
```



Definition:

Closure = Calling inside function outside the program.



Assignment 11

1. Create closure that returns square function.
 2. Create closure that returns multiplication function.
-



13 Decorator (Very Important ★★★)

Definition:

Without modifying main function, adding extra functionality.

Real-Time Example:

Before payment → Must login.

Main function = Payment

Decorator = Login check

```
def decorator_func(func):  
    def wrapper():  
        print("Login Successful")  
        func()  
    return wrapper
```

```
@decorator_func  
def payment():  
    print("Payment Done")
```

```
payment()
```

Output:

```
Login Successful  
Payment Done
```



Assignment 12

1. Create decorator to check zero in list before multiplication.
 2. Create decorator to check division by zero.
 3. Create decorator to check login before checkout.
-



Generator (Introduction)

Uses `yield` keyword.

```
def count():  
    yield 1  
    yield 2  
    yield 3
```

```
for i in count():  
    print(i)
```



Assignment 13

1. Create generator for even numbers.
 2. Create generator for Fibonacci series.
-



Important Concepts to Revise

- Return Flow
 - Nested Function
 - Closure
 - Decorator
 - `*args` and `**kwargs`
 - Default arguments
-



Final Practice Questions

1. Create login decorator without modifying main function.
 2. Handle ZeroDivisionError using decorator.
 3. Create closure for bank interest calculator.
 4. Create function to calculate product of list.
 5. Write program using *args and **kwargs together.
-



Homework (Exam Level)

1. Write decorator to prevent zero in product list.
2. Write closure to generate multiplication table.
3. Create generator to generate squares of numbers.
4. Create profile system using **kwargs.
5. Create calculator using all types of functions